

矩阵与张量运算

续本达

清华大学 工程物理系

2025-06-30

下载 `asym.py`

<https://git.tsinghua.edu.cn/physics-data/lecture/-/blob/master/asym.py>

保存到课堂练习的路径

模块是一系列相关的变量、函数和类的集合，可以由程序复用。使用 `import`, `from`, `as` 等调用

```
import fractions
import fractions as frcs
from fractions import Fraction
from fractions import Fraction as Frc
```

模块的例子

`fractions` 无限精度分数

`decimal` 十进制小数

`itertools` 迭代器

`NumPy` 数值计算

`SciPy` 科学计算

单 Python 脚本式模块

```
from asym import eps
```

```
eps(4).shape
```

4 4 4 4

- 文本文件在 Python 中被抽象成迭代器。
- 使用 with 让与外部资源交互（如文件）的代码更简洁。

```
from collections import Counter
with open("departments.csv") as f_input:
    print(Counter(f_input))
```

Counter({'致理书院\n': 30, '工物系\n': 20, '未央书院\n': 20, '物理系\n': 7, '上海交大\n': 1, '数学系\n': 1, '探微书院\n': 1})

- 数组
 - 与元组、列表的异同
 - 创建
 - 下标的范围
 - 下标的扩展
 - 替代循环

```
import numpy as np
```

```
n = np.arange(10)
```

```
n**2
```

0 1 4 9 16 25 36 49 64 81

- 矩阵乘法是张量缩并的特例

$$\sum_j a_{ij} b_{jk} = c_{ik}$$

- 元素乘法是逐个数字相乘

$$a_{ij} b_{ij} = c_{ij}$$

```
a = b = np.ones((2,2))
print(a * b)
print(a @ b)
```

```
[[1. 1.]
 [1. 1.]]
[[2. 2.]
 [2. 2.]]
```

- 矩阵可表示为两个下标的矩阵元：二阶张量 a_{ij} 。
 - 三个下标：三阶张量 b_{ijk} 。
 - 一个下标：向量 x_i 。
 - 无下标：标量。

- `tensordot` , 可标注哪些下标进行缩并

```
a = np.arange(60.).reshape(3,4,5)
b = np.arange(24.).reshape(4,3,2)
np.tensordot(a,b, axes=([1,0],[0,1]))
```

4400 4730

4532 4874

4664 5018

4796 5162

4928 5306

- einsum

```
a = np.arange(25).reshape(5,5)
```

```
b = np.arange(5)
```

```
np.einsum('ii', a) # 取迹
```

60

```
np.einsum('ij->i', a) # 对 'j' 方向求和
```

10 35 60 85 110

```
np.einsum('ij,j', a, b) # 矩阵乘法
```

30 80 130 180 230

```
a = np.arange(60.).reshape(3,4,5)
b = np.arange(24.).reshape(4,3,2)
np.einsum('ijk,jil->kl', a, b)
```

4400	4730
4532	4874
4664	5018
4796	5162
4928	5306

灵活缩并

```
a = np.ones(64).reshape(2,4,8)
np.einsum('ijk,ilm,njm,nlk',a,a,a,a)
```

4096.0

当代码语法的描述能力不断变强时，可能比数学公式还简单。

$$\sum_{i=1}^I \sum_{j=1}^J \sum_{k=1}^K \sum_{l=1}^L \sum_{m=1}^M \sum_{n=1}^N a_{ijk} a_{ilm} a_{njm} a_{nlk}$$

- 经典的 formula translation (Fortran 语言)，先推导再转成程序。
 - 可能用程序来描述公式的推导吗？
 - 程序帮助我们推导公式？

- 在量子力学中为自旋 $\frac{1}{2}$ 态空间下的角动量算符表示。

$$\sigma_1 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \sigma_2 = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \sigma_3 = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

- 科学发展的顺序和培养方案的顺序不同。Heisenberg 发明矩阵力学时还未掌握矩阵。
 - 即使尚未掌握量子力学也可以熟悉 Pauli 矩阵并进行计算。

$$\sigma_1 \sigma_2 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} = \begin{pmatrix} i & 0 \\ 0 & -i \end{pmatrix} = i \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = i \sigma_3$$

同理,

$$\sigma_2 \sigma_1 = -i \sigma_3$$

$$\sigma_2 \sigma_3 = i \sigma_1$$

$$\sigma_3 \sigma_2 = -i \sigma_1$$

$$\sigma_3 \sigma_1 = i \sigma_2$$

$$\sigma_1 \sigma_3 = -i \sigma_2$$

- 构造三个 Pauli 矩阵

```
pauli = []  
pauli.append(np.array([0, 1, 1, 0]).reshape(2, 2))  
pauli.append(np.array([0, -1j, 1j, 0]).reshape(2, 2))  
pauli.append(np.array([1, 0, 0, -1]).reshape(2, 2))  
for m in pauli:  
    print(m)
```

```
[[0 1]  
 [1 0]]  
[[ 0.+0.j -0.-1.j]  
 [ 0.+1.j  0.+0.j]]  
[[ 1  0]  
 [ 0 -1]]
```

```
for m in pauli:  
    print("trace is {}".format(np.trace(m)),  
          "eigenvalues are: ", np.linalg.eigvals(m))
```

```
trace is 0; eigenvalues are: [ 1. -1.]  
trace is 0j; eigenvalues are: [ 1.+0.j -1.+0.j]  
trace is 0; eigenvalues are: [ 1. -1.]
```

```
def commute(a,b):  
    '''  
        commutation operator: ab-ba  
    '''  
    return a@b - b@a  
  
for i in range(3):  
    l = (i+1) % 3  
    m = (i+2) % 3  
    if np.all(commute(pauli[i], pauli[l]) == 2j * pauli[m]):  
        print(f"[ pauli_{i} , pauli_{l} ] == 2i pauli_{m}")
```

```
[ pauli_0 , pauli_1 ] == 2i pauli_2  
[ pauli_1 , pauli_2 ] == 2i pauli_0  
[ pauli_2 , pauli_0 ] == 2i pauli_1
```

彻底避免使用列表和 for 循环

- 构造 Pauli 矩阵时使用了列表和 for 循环，不够简洁
- 直接构造张量计算，省去交换关系验证中的循环

将 Pauli 矩阵的下标当成高一阶的张量

$$[\sigma_i, \sigma_j] = 2i\epsilon_{ijk}\sigma^k$$

- (ϵ_{ijk}) 是全反称张量，指标有相同时为 0，指标奇排列为 -1，偶排列为 1。

```
from asym import eps  
eps(3)
```

```
array([[ [ 0.,  0.,  0.],  
        [ 0.,  0.,  1.],  
        [ 0., -1.,  0.]],
```

```
       [[ 0.,  0., -1.],  
        [ 0.,  0.,  0.],  
        [ 1.,  0.,  0.]],
```

```
       [[ 0.,  1.,  0.],
```

$$[\sigma_i, \sigma_j] = 2i\epsilon_{ijk}\sigma^k$$

```
import numpy as np
pauli = np.array([np.array([(0,1),(1,0)]),
                  np.array([(0,-1j),(1j,0)]),
                  np.array([(1,0),(0,-1)])])
pp = np.einsum('ixy,jyz->ijxz', pauli, pauli)
commutate = pp - np.transpose(pp, (1,0,2,3))
np.all(commutate == 2j * np.tensordot(eps(3), pauli, axes=([2], [0])))
```

True

$$\{\sigma_i, \sigma_j\} = 2\delta_{ij}I$$

```
ac = pp + np.transpose(pp, (1,0,2,3))  
np.all(ac == 2 * np.einsum('ij,xz->ijxz', np.eye(3), np.eye(2)))
```

- 掌握物理学原理：这些形式化的验证，让我们无跳跃步骤地 **彻底** 完成推导。

复习

乘法

Pauli 矩阵

Dirac 矩阵

备用

$$\gamma^0 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}, \quad \gamma^1 = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & -1 & 0 & 0 \\ -1 & 0 & 0 & 0 \end{pmatrix}$$
$$\gamma^2 = \begin{pmatrix} 0 & 0 & 0 & -i \\ 0 & 0 & i & 0 \\ 0 & i & 0 & 0 \\ -i & 0 & 0 & 0 \end{pmatrix}, \quad \gamma^3 = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \\ -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

$$\{\gamma^\mu, \gamma^\nu\} = 2\eta^{\mu\nu} I$$

- $\{A, B\} = AB + BA$ 是 反对易子
- $\eta^{\mu\nu}$ 是闵可夫斯基度规 `np.diag((1, -1, -1, -1))`

- 定义 $\gamma^5 \equiv i\gamma^0\gamma^1\gamma^2\gamma^3$
- $\gamma_\mu = \eta_{\mu\nu}\gamma^\nu$
 - $\eta_{\mu\nu}$ 也是闵可夫斯基度规 `np.diag((1, -1, -1, -1))`

NumPy 编程验证:

- γ^5 是厄米阵
- $(\gamma^5)^2 = I$
- $\{\gamma^5, \gamma^\nu\} = 0$
- $\gamma^\nu\gamma_\nu = 4I$
- $\gamma^\mu\gamma^\nu\gamma^\rho\gamma_\mu = 4\eta^{\nu\rho}I$
- $\text{Tr}(\gamma^\mu) = 0$
- $\text{Tr}(\gamma^\mu\gamma^\nu) = 4\eta^{\mu\nu}$
- $\text{Tr}(\gamma^5) = 0$

```
import numpy as np
pauli = np.array([[0,1],[1,0]],
                  [[0,-1j],[1j,0]],
                  [[1,0],[0,-1]])
gamma0 = np.diag((1,1,-1,-1))
gamma2p = np.diag((-1j,1j,1j,-1j))
dirac = np.array((gamma0,
                  gamma0[:, ::-1],
                  gamma2p[:, ::-1],
                  np.roll(np.diag((1,-1,-1,1)), 2, axis=1)))
```

验证 Dirac 矩阵的反对易关系

$$\{\gamma^\mu, \gamma^\nu\} = 2\eta^{\mu\nu} I$$

```
def anticomm(x, y):
    '''
    反对易子操作
    =====
    传入 x 和 y 张量,
    其后两阶可以解读为矩阵。

    返回, xy + yx 的张量值。
    '''
    return x @ y + y @ x

mkov = np.diag((1, -1, -1, -1))
np.all(anticomm(dirac[None], dirac[:, None]) ==
        2 * np.einsum('ij,xz->ijxz', mkov, np.eye(4)))
```

True

```
gamma5 = 1j * dirac[0] @ dirac[1] @ dirac[2] @ dirac[3]  
np.all(np.conj(gamma5.T) == gamma5)
```

True

$$(\gamma^5)^2 = I$$

```
np.all(gamma5 @ gamma5 == np.eye(4))
```

True

复习

乘法

Pauli 矩阵

Dirac 矩阵

备用

$$\{\gamma^5, \gamma^\nu\} = 0$$

```
np.all(anticomm(gamma5[None], dirac) == 0)
```

True

$$\gamma^\nu \gamma_\nu = 4I$$

```
np.all(np.einsum('ixy,ij,jyz->xz', dirac, mkov, dirac) == 4 * np.eye(4))
```

True

$$\text{Tr}(\gamma^\mu \gamma^\nu) = 4\eta^{\mu\nu}$$

```
np.all(np.einsum('ixy,jyx', dirac, dirac) == 4 * mkov)
```

True

$$\gamma^\mu \gamma^\nu \gamma^\rho \gamma_\mu = 4\eta^{\nu\rho} I$$

```
np.all(np.einsum('xab,jbc,kcd,xy,yde->jkae', dirac,dirac,dirac,mkov,dirac)
      == 4 * np.einsum('ij,xy->ijxy', mkov, np.eye(4)))
```

True

$$\text{Tr}(\gamma^\mu) = 0$$

```
np.all(np.einsum('ixx', dirac) == 0)
```

True

$$\text{Tr}(\gamma^5) = 0$$

```
np.trace(gamma5)
```

0j

- 希望可以多一点写代码的指导
课上的例子是写代码的指导。如果不够，同学之间和小助教多讨论切磋。
- 希望自己学好，谢谢助教和老师（约占 40%）
好！我们一起努力克服困难。
- 希望学到更多对实验物理研究有用的工具
好！这是本人十几年来一直不变的初心。
- 希望作业能简单一点，分高一点
作业的难度和给分高低服务于课程目标的设计。
世界上本没有水课，太简单分太高就成了水课。

同学们的建议反馈（一）

- 希望可以多一点写代码的指导
课上的例子是写代码的指导。如果不够，同学之间和小助教多讨论切磋。
- 希望自己学好，谢谢助教和老师（约占 40%）
好！我们一起努力克服困难。
- 希望学到更多对实验物理研究有用的工具
好！这是本人十几年来一直不变的初心。
- 希望作业能简单一点，分高一点
作业的难度和给分高低服务于课程目标的设计。
世界上本没有水课，太简单分太高就成了水课。

- 希望可以多一点写代码的指导
课上的例子是写代码的指导。如果不够，同学之间和小助教多讨论切磋。
- 希望自己学好，谢谢助教和老师（约占 40%）
好！我们一起努力克服困难。
- 希望学到更多对实验物理研究有用的工具
好！这是本人十几年来一直不变的初心。
- 希望作业能简单一点，分高一点
作业的难度和给分高低服务于课程目标的设计。
世界上本没有水课，太简单分太高就成了水课。

- 希望可以多一点写代码的指导
课上的例子是写代码的指导。如果不够，同学之间和小助教多讨论切磋。
- 希望自己学好，谢谢助教和老师（约占 40%）
好！我们一起努力克服困难。
- 希望学到更多对实验物理研究有用的工具
好！这是本人十几年来一直不变的初心。
- 希望作业能简单一点，分高一点
作业的难度和给分高低服务于课程目标的设计。
世界上本没有水课，太简单分太高就成了水课。

同学们的建议反馈（二）

- 我实在是太菜了，助教总是凶巴巴，可以温柔一点吗？
有些人明白他们不该粗鲁或傲慢的提问并要求得到答复，但他们选择另一个极端 —— 低声下气：我知道我只是个可悲的新手，一个撸瑟，但…。这既使人困扰，也没有用，尤其是伴随着与实际问题的描述时更令人反感。
别用原始灵长类动物的把戏来浪费你我的时间。取而代之的是，尽可能清楚地描述背景条件和你的问题情况。这比低声下气更好地定位了你的位置。
—— Eric Raymond，提问的智慧

- 解决未知问题是本课程的教学目标，也是数据时代黑客技能的基本组成部分，不要放弃宝贵的学习机会。
- 希望同学在提问之前，做一些努力，尝试先自己解决问题。
- 查找 FAQ 和群聊天记录，可能已经有人遇到并解决了同样问题。
- 提问时，把问题描述清楚，不要说自己“菜”“弱”。

课堂豁免

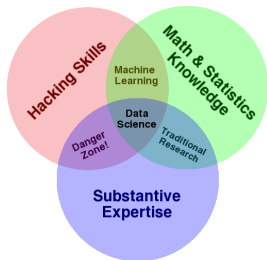
- 在课堂上，没有时间作系统调研，以上规则不适用。
- 课堂上遇到疑惑，请你大声发问！这也是在帮助同学和老师。

同学们的建议反馈（三）

- 作业难度跳跃太大是这样的。
- 从以往的大作业项目看，强烈建议大作业中给出更加详细的公式、推导过程和计算方法，以便没有选修过相关课程的同学可以更好地理解项目的背景知识、数值计算方法等。期待不要把作业中没有讲的东西都扔给自学。参考老师理解的给分梯度，请在遇到知识盲点时，通过自学补足。大作业的设计目标是模拟科研环境，客观世界从不在乎你已经学过什么。
- 希望讲授的时候能够慢一些（约占 15%）请在课堂上遇到困难时直接发问，本人会加入更多的反馈机制。
- 希望讲授的时候能够快一些（约占 10%）请帮助周围的同学，或直接去解决大作业中的开放问题。

同学们的建议反馈（四）

- 希望能简单的列出之后的上课大纲，方便同学们预习好。
- 期待马上上手机器学习，希望学习神经网络



- 炒菜式的机器学习非常简单，不用上课也可以学会。
- 有了 Hacking skills 和数理统计的功底，机器学习才能发挥出作用。

同学们的建议反馈（五）

- 希望详细演示下 debug 的过程
好！
- 希望了解更多的 linux 的骚气操作
好！第三周。
- 希望老师多讲解每个知识点的具体应用的意义。少讲语法，突出物理
好！
- 建议作业和课件在课程开始时就全部发放
课件可以，作业课后发。
- 将所有同学问过的问题整理到某个地方。
 - <https://physics-data.meow.plus/faq/>
 - 你学习如何通过关键词提示找到答案
- 希望老师多举一些例子，整理出来发给大家。优秀代码分享。
好！

如何通过关键词提示找到答案

- 搜索引擎的用法演示
<https://zh.lmgtfy.com/>
<https://www.google.com/>
<http://www.bing.com/>
<https://duckduckgo.com/>
其它的搜索引擎都强烈不推荐。

- 如果你的问题被人无视了，请对号入座：
- 问题：我能在哪找到 X 程序或 X 资源？
 - 回答：就在我找到它的地方啊，白痴 —— 搜索引擎的那一头。天哪！难道还有人不会用 Google 吗？
- 问题：我怎样用 X 做 Y？
 - 回答：如果你想解决的是 Y，提问时别给出可能并不恰当的方法。这种问题说明提问者不但对 X 完全无知，也对 Y 要解决的问题糊涂，还被特定形势禁锢了思维。最好忽略这种人，等他们把问题搞清楚了再说。
- 问题：如何设定我的 shell 提示??
 - 回答：如果你有足够的智慧提这个问题，你也该有足够的智慧去 RTFM，然后自己去找出来。
- 问题：我可以用 Bass-o-matic 文件转换工具将 AcmeCorp 档案转换为 $\text{T}_\text{E}\text{X}$ 格式吗？
 - 回答：试试看就知道了。如果你试过，你既知道了答案，就不用浪费我的时间了。

- 问题：我的{程序/设定/SQL 语句}不工作
 - 回答：这不算是问题吧，我对要我问你二十个问题才找得出你真正问题的问
题没兴趣 —— 我有更有意思的事要做呢。在看到这类问题的时候，我的反应
通常不外如下三种
 - 你还有什么要补充的吗？
 - 真糟糕，希望你能搞定。
 - 这关我有什么屁事？

- 具体问题，请到
<https://git.tsinghua.edu.cn/physics-data/faq/-/issues/>
方便遇到类似问题的同学快速找到。
- 当你的问题得到解答后，助教可能觉得此问题非常有价值。请配合助教发 Pull Request，将它收集到 <https://physics-data.meow.plus/faq/>。